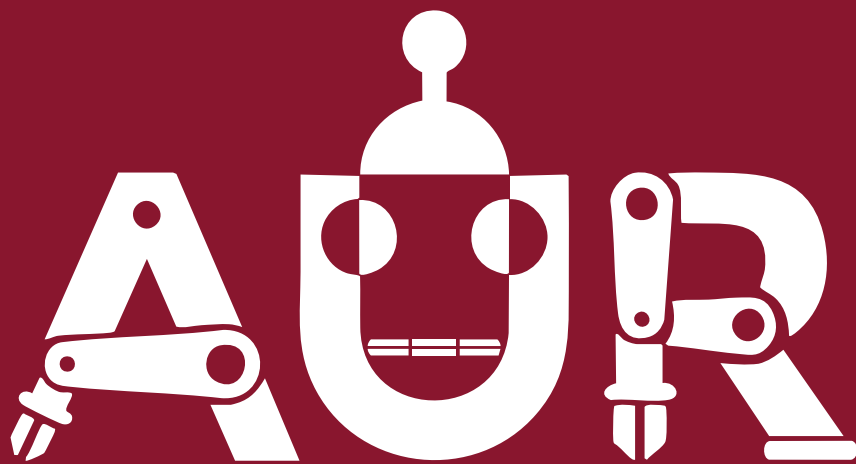




# Training '26

## Software | Phase II

### ROS2 Workshop 1 Extension



## Introduction to ROS 2 Topics

Before we dive into the tasks, let's review the fundamental concepts you will be using today. You already know that ROS 2 nodes communicate using **Topics**. Here is a quick refresher on the components we will use:

- **Publishers:** Send data (messages) to a topic.
- **Subscribers:** Listen for data on a topic and trigger a callback function when new data arrives.
- **Messages:** The standard data structures used. Today we will use Twist (from `geometry_msgs.msg`) to send velocity commands, and Pose (from `turtlesim.msg`) to read the robot's current position and orientation.
- **Timers:** Used to execute a function repeatedly at a specific frequency.

## Task 1: Basic Turtle Controller

**Goal:** Write a node that continuously drives the turtle in a circle while logging its live coordinates to the terminal.

### 1 Guidance & Architecture

For this task, you need to create a custom node called TurtleController.

- **Publisher:** Send Twist velocity commands to `/turtle1/cmd_vel`.
- **Subscriber:** Listen to Pose messages on `/turtle1/pose` and log the x, y, and theta values.
- **Timer:** Call a velocity publishing function at 2 Hz (every 0.5 seconds).

### 2 Pseudocode Guidance

```

1
2 CLASS TurtleController INHERITS Node:
3     FUNCTION init():
4         Initialize node named 'turtle_controller_node'
5         Create publisher for Twist messages on '/turtle1/cmd_vel'
6         Create subscriber for Pose messages on '/turtle1/pose' calling pose_callback
7         Create timer running at 2 Hz calling publish_velocity
8
9     FUNCTION pose_callback(msg):
10        Log turtle telemetry: X, Y, Theta from msg
11
12    FUNCTION publish_velocity():
13        Create Twist message
14        Set linear velocity (X = 2.0 m/s)
15        Set angular velocity (Z = 1.0 rad/s)
16        Publish Twist message
17
18
19    FUNCTION main():
20        Initialize rclpy
21        Instantiate TurtleController node
22        Spin node
23        Clean up and shutdown rclpy

```



## Task 2: Go-To-Goal Proportional Controller

**Goal:** Write an advanced node that actively navigates the turtle to a specific target coordinate on the screen using proportional control feedback.

### 1 Guidance & Architecture

In this task, you will implement a closed-loop controller inside a node called `GoToGoalNode` to reach target coordinate  $(10.0, 10.0)$ :

- **Control Frequency:** Run the main control timer loop at 20 Hz (every 0.05 seconds).
- **Distance Error:** Calculate Euclidean distance  $\sqrt{x^2 + y^2}$ . Stop if error  $< 0.1$ .
- **Heading Error:** Calculate desired angle using  $\text{atan2}(y, x)$  and normalize heading error to be ranged from  $[-\pi, \pi]$  instead of being  $< -\pi$  or  $> \pi$ .
- **Proportional Logic:** If heading error  $> 0.05$  rad, turn in place ( $v_x = 0$ ). Otherwise, drive forward with proportional linear speed (capped at 2.0 m/s) while correcting heading.

### 2 Pseudocode Guidance

```

1
2  DEFINE TARGET_X = 10.0, TARGET_Y = 10.0
3
4  CLASS GoToGoalNode INHERITS Node:
5      FUNCTION **init**():
6          Initialize node named 'go_to_goal_node'
7          Set proportional gains (kp_linear = 1.5, kp_angular = 6.0)
8          Set tolerances (distance_tolerance = 0.1, angle_tolerance = 0.05)
9          Create publisher for Twist on '/turtle1/cmd_vel'
10         Create subscriber for Pose on '/turtle1/pose' calling pose_callback
11         Create control loop timer at 20 Hz calling control_loop
12
13     FUNCTION pose_callback(msg):
14         Update current_pose with msg data
15
16     FUNCTION normalize_angle(angle):
17         Keep angle wrapped within [-pi, pi]
18         RETURN normalized angle
19
20     FUNCTION control_loop():
21         IF current_pose is None OR goal_reached THEN RETURN
22
23         // 1. Calculate errors
24         dx = TARGET_X - current_pose.x
25         dy = TARGET_Y - current_pose.y
26         distance_error = sqrt(dx^2 + dy^2)
27
28         target_angle = atan2(dy, dx)
29         heading_error = normalize_angle(target_angle - current_pose.theta)
30

```



```
31     // 2. Check completion
32     IF distance_error < distance_tolerance THEN:
33         Publish zero velocity
34         Set goal_reached = True
35         RETURN
36
37     // 3. Control logic
38     IF abs(heading_error) > angle_tolerance THEN:
39         // Align orientation first
40         linear_x = 0.0
41         angular_z = kp_angular * heading_error
42     ELSE:
43         // Move forward and adjust heading concurrently
44         linear_x = min(kp_linear * distance_error, 2.0)
45         angular_z = kp_angular * heading_error
46
47     Publish velocity command with (linear_x, angular_z)
48
49
50 FUNCTION main():
51     Initialize rclpy
52     Instantiate GoToGoalNode
53     Spin node
54     Clean up and shutdown rclpy
```